

Ricostruzione e georeferenziazione di traiettorie da video

Lorenzo Maria Alberto Paoria

Matricola: 1000016111

Progetto di Analisi Numerica

Prof. Sebastiano Boscarino

1 Obiettivo e dati

Il progetto confronta i metodi di interpolazione del corso nella ricostruzione delle posizioni mancanti di automobili riprese da telecamere fisse. YOLO11n rileva gli oggetti; ByteTrack associa le rilevazioni nel tempo. Il punto della traiettoria è il *bottom center* della box:

$$u = \frac{u_{\min} + u_{\max}}{2}, \quad v = v_{\max}, \quad P(t) = \begin{bmatrix} u(t) \\ v(t) \end{bmatrix}. \quad (1.1)$$

Le box sono dati del rilevatore utilizzati per localizzare l'oggetto; la grandezza da ricostruire è la traiettoria.

Tabella 1: Video Urban Tracker elaborati per intero [2].

Video	Frame	Durata circa	Osservazioni YOLO/ByteTrack
Sherbrooke	4 000	2 min 13 s	40 331
René-Lévesque	8 501	4 min 44 s	219 327

Stato del lavoro.

- Confronto in pixel, esperimenti sintetici, clip e video completi: eseguiti.
- Georeferenziazione: calibrazioni manuali pubblicate e primo confronto in metri eseguito su 406 gap nella zona valida; mancano checkpoint geografici indipendenti.
- Cinematica: valutazione eseguita su 406 gap metrici; il riferimento cinematico è una stima da differenze finite della traiettoria completa YOLO/ByteTrack e non una ground truth indipendente.

Le osservazioni complete sono salvate; alcune vengono poi nascoste agli interpolatori e usate soltanto dal valutatore. Il riferimento è quindi YOLO/ByteTrack, non una posizione fisica esatta. Il protocollo è *post-tracking*: gli identificatori sono già assegnati prima della rimozione dei campioni.

2 Concetti del corso utilizzati

Questo capitolo riassume gli strumenti di Analisi Numerica impiegati nel progetto. Il capitolo successivo ne presenta formule, condizioni al contorno ed estratti di implementazione.

2.1 Strumenti spiegati nelle dispense

Questa sezione raccoglie i concetti direttamente riconducibili alle dispense del corso: interpolazione polinomiale, spline, nodi di Chebyshev, periodicità e interpolazione trigonometrica. Le pagine precise sono richiamate nel capitolo successivo.

2.1.1 Interpolazione di dati discreti

Il problema centrale è ricostruire i valori di una traiettoria nei tempi in cui l'osservazione è stata rimossa. Dati nodi distinti $(x_i, y_i)_{i=0}^n$, si cerca una funzione $\tilde{f}$ tale che

$$\tilde{f}(x_i) = y_i, \quad i = 0, \dots, n.$$

Nel progetto x_i è il tempo e y_i è una coordinata dell'automobile, prima in pixel (u, v) e poi, quando la calibrazione è valida, in metri (E, N) . Il riferimento completo viene conservato prima della mascheratura: i valori nascosti servono solo a misurare l'errore.

2.1.2 Rappresentazioni polinomiali

Sono state utilizzate le tre rappresentazioni polinomiali affrontate nel corso: base monomiale e matrice di Vandermonde, forma di Lagrange e forma di Newton con differenze divise e schema di Horner. Con gli stessi nodi e dati esse rappresentano lo stesso polinomio di grado al più n ; il confronto separa quindi l'errore di interpolazione dalla formulazione numerica.

2.1.3 Spline e condizioni al contorno

Per limitare le oscillazioni dei polinomi globali sono state considerate S1 lineare a tratti, S2 quadratica a tratti, S3 naturale, S3 vincolata e S3 periodica. Le spline cubiche sono C^2 ; la S3 naturale annulla le derivate seconde agli estremi, mentre la vincolata assegna le derivate prime. La periodicità richiede valori e derivate compatibili alla chiusura del periodo. Le condizioni al bordo modificano anche le derivate usate per la cinematica.

2.1.4 Nodi, periodicità e interpolazione trigonometrica

L'esperimento di Runge mostra che i nodi equispaziati possono produrre oscillazioni elevate, mentre i nodi di Chebyshev riducono l'errore massimo. Chebyshev è una scelta dei nodi, non una nuova formulazione sugli stessi dati. Per il segnale sintetico periodico è stata usata anche l'interpolazione trigonometrica, cioè una combinazione finita di seni e coseni; con campioni mancanti i coefficienti sono ricavati risolvendo il sistema sui soli tempi osservati. La periodicità non è imposta ai video stradali.

2.2 Estensioni introdotte nel progetto

La spline quadratica S2 e la costruzione baricentrica di Floater–Hormann non sono presentate come metodi autonomi principali nelle dispense. S2 è stata costruita a partire dalla definizione generale di spline $s^m \in C^{m-1}$, mentre Floater–Hormann è un algoritmo specifico della famiglia razionale introdotta nel corso. Sono state aggiunte per ampliare il confronto sperimentale e sono indicate come estensioni anche nelle tabelle dei risultati.

2.3 Strumenti applicativi e criteri di valutazione

2.3.1 Normalizzazione, stabilità e condizionamento

I tempi sono trasformati nell'intervallo $[-1, 1]$, migliorando la stabilità del sistema di Vandermonde e rendendo confrontabili i supporti. Per la base monomiale si registra il numero di condizionamento $\kappa_2(V_n)$, che misura la sensibilità rispetto a perturbazioni e arrotondamenti. La scelta della formulazione non elimina invece l'errore dovuto a un supporto troppo ampio o a troppi nodi.

2.3.2 Derivazione e grandezze cinematiche

Velocità e heading non sono interpolati separatamente: si derivano i modelli delle coordinate,

$$v_E(t) = E'(t), \quad v_N(t) = N'(t), \quad V(t) = \sqrt{v_E(t)^2 + v_N(t)^2},$$

$$\theta(t) = \left[\frac{180}{\pi} \operatorname{atan2}(v_E(t), v_N(t)) + 360 \right] \bmod 360.$$

Le accelerazioni sono le seconde derivate. La regola della catena per i tempi normalizzati e la regolarità della spline sono quindi parte del metodo numerico.

2.3.3 Metriche di errore e confronto

La valutazione usa i campioni nascosti e due metriche di errore complementari. Posto $e_i = \|P_i - \hat{P}_i\|_2$, dove P_i è la posizione di riferimento e $\hat{P}_i$ quella ricostruita, si definiscono

$$\text{ADE} = \frac{1}{M} \sum_{i=1}^M e_i, \quad \text{RMS}_P = \sqrt{\frac{1}{M} \sum_{i=1}^M e_i^2}. \quad (2.1)$$

ADE (*Average Displacement Error*) è l'errore medio di posizione: per ogni campione nascosto si calcola la distanza euclidea tra la posizione ricostruita e quella di riferimento, poi si fa la media. Mantiene le stesse unità della posizione, cioè pixel oppure metri, e può essere interpretato come errore tipico della ricostruzione.

RMS (*Root Mean Square*) è la radice della media dei quadrati degli errori di posizione. Ha anch'essa unità pixel oppure metri, ma penalizza maggiormente gli errori grandi perché questi vengono elevati al quadrato prima della media. È quindi utile per mettere in evidenza oscillazioni o pochi errori particolarmente elevati. Vale sempre $\text{RMS}_P \geq \text{ADE}$, con uguaglianza quando tutti gli errori hanno la stessa ampiezza.

Nel progetto ADE e RMS sono calcolati sugli stessi campioni nascosti e rispetto allo stesso riferimento YOLO/ByteTrack. Non sono quindi misure assolute dell'errore fisico dell'automobile o della precisione GPS. Sono inoltre registrati tempi di fit, fallimenti numerici, condizionamento e copertura. L'errore di ricostruzione è tenuto distinto dall'errore della calibrazione geografica.

Omografia, DLT, SVD e conversione WGS84 appartengono invece alla calibrazione del piano stradale, descritta nel capitolo successivo, e non sono metodi di interpolazione del corso. Analogamente, Python, NumPy, SciPy e i video sono strumenti di implementazione e di verifica del progetto, non contenuti teorici aggiuntivi delle dispense.

3 Calibrazione del piano stradale

3.1 Orientamento sulla mappa

I link geografici pubblicati dal dataset permettono di centrare la mappa sulle scene. Le coordinate seguenti sono **centri approssimativi delle inquadrature**, non posizioni misurate delle telecamere né punti di controllo da usare automaticamente nel fit.

Scena	Latitudine	Longitudine
Sherbrooke / Atateken, ex Amherst	45,521379	-73,565683
René-Lévesque Ouest, zona 1425	45,494921	-73,574007

3.2 Omografia

Un'omografia associa punti dell'immagine a punti di un unico piano. La calibrazione usa pixel normalizzati e restituisce coordinate locali nell'ordine Nord-Est:

$$\begin{bmatrix} \tilde{N} \\ \tilde{E} \\ q \end{bmatrix} = \mathbf{H} \begin{bmatrix} u/W \\ v/H_{\text{img}} \\ 1 \end{bmatrix}, \quad N = \frac{\tilde{N}}{q}, \quad E = \frac{\tilde{E}}{q}, \quad q \neq 0. \quad (3.1)$$

W, H_{img} sono le dimensioni del frame. Il progetto usa campi nominati e fornisce agli interpolatori la coppia (E, N) in metri. Latitudine e longitudine WGS84 sono ottenute con la conversione locale della stessa libreria.

Procedura operativa.

1. Selezionare alcuni punti riconoscibili sul piano stradale nell'immagine e identificarne le coordinate geografiche o metriche corrispondenti.
2. Usare almeno quattro coppie non degeneri, preferibilmente 6–10 punti ben distribuiti nell'area visibile.
3. Stimare la matrice omografica con DLT normalizzato e SVD, controllando i residui e individuando eventuali outlier.
4. Verificare la calibrazione su punti di controllo non utilizzati nella stima, quindi salvare la matrice e i parametri della scena.

Sono stati pubblicati 7 punti per Sherbrooke, di cui 6 inlier, con RMS del fit di circa 0,280 m, e 4 punti per René-Lévesque, con residuo numericamente nullo. Non sono ancora stati forniti checkpoint indipendenti.

Python – estratto essenziale. L'adattatore richiama l'API condivisa `GroundProjector`:

```
from src.georeference import GroundCalibration, settings

g = GroundCalibration(settings(), "urban_sherbrooke", 800, 600)
p = g.project((u, v))
E, N = p["east_m"], p["north_m"]
lat, lon = p["latitude_deg"], p["longitude_deg"]
```

Il fit condiviso usa DLT normalizzato, SVD e selezione robusta delle corrispondenze. La proiezione controlla sorgente, preprocessing, dimensioni e poligono valido. Si considerano soltanto punti del piano stradale calibrato: tetti, facciate e superfici a quote diverse non sono descritti dalla stessa omografia.

Il flusso metrico è:

$$\text{pixel} \longrightarrow (E, N) \text{ in metri} \longrightarrow \text{interpolazione} \longrightarrow (\text{latitudine}, \text{longitudine}).$$

Il residuo sui punti del fit e l'errore su punti indipendenti devono essere distinti. Quattro corrispondenze possono dare residuo quasi nullo senza certificare l'accuratezza dell'intera zona.

4 Metodi: definizione, formula e implementazione

Si mantiene la notazione delle dispense: x_i sono i **nodi** e y_i i valori della coordinata da interpolare. Nel progetto x_i corrisponde al tempo e y_i a u_i, v_i oppure E_i, N_i . Le **condizioni di interpolazione** sono

$$\tilde{f}(x_i) = y_i, \quad i = 0, \dots, n. \quad (4.1)$$

Gli estratti mostrano fit e valutazione dal modulo `src/interpolation.py`. Controlli degli input, gestione degli errori e derivate sono omessi dagli estratti.

4.1 Preparazione comune dei tempi

Il tempo viene normalizzato in $[-1, 1]$:

$$\tau = 2 \frac{t - t_{\min}}{t_{\max} - t_{\min}} - 1. \quad (4.2)$$

Nei frammenti, `x` contiene i nodi normalizzati e `xq` i tempi normalizzati da ricostruire. `y` contiene soltanto valori visibili. La trigonometrica usa invece direttamente `t` e `tq`, con periodo fisico noto.

```
import numpy as np
from numpy.polynomial import Polynomial
from scipy.interpolate import CubicSpline, PPoly, make_interp_spline

span = t[-1] - t[0]
x = 2 * (t - t[0]) / span - 1
xq = 2 * (tq - t[0]) / span - 1
```

4.2 Interpolazione polinomiale nella base dei monomi

Dispense di Interpolazione, pp. 7–9 [1].

Si determina un polinomio risolvendo il sistema con **matrice di Vandermonde**:

$$p_n(x) = \sum_{j=0}^n a_j x^j, \quad V_n a = y, \quad (V_n)_{ij} = x_i^j. \quad (4.3)$$

Con $n + 1$ nodi distinti esiste un unico polinomio di grado al più n .

Python – estratto essenziale. `numpy.vander`, `numpy.linalg.solve`, `Polynomial`:

```
V = np.vander(x, N=len(x), increasing=True)
a = np.linalg.solve(V, y)
p = Polynomial(a)
yq = p(xq)
```

4.3 Polinomi di Lagrange

Dispense di Interpolazione, pp. 10–12 [1].

Il polinomio è una combinazione lineare dei valori y_i e dei polinomi elementari L_i :

$$p_n(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (4.4)$$

Python – estratto essenziale. Formula diretta, implementata mediante `numpy.prod`:

```
yq = np.zeros_like(xq, dtype=float)
for i in range(len(x)):
    other = np.delete(x, i)
    denominator = np.prod(x[i] - other)
    Li = np.prod(xq[:, None] - other, axis=1) / denominator
    yq += y[i] * Li
```

4.4 Forma di Newton e differenze divise

Dispense di Interpolazione, pp. 18–27 [1].

Si costruisce il polinomio mediante **differenze divise** e lo si valuta con lo **schema di Horner**:

$$f[x_i] = y_i, \quad f[x_0, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}, \quad (4.5)$$

$$P_n(x) = f[x_0] + \sum_{k=1}^n f[x_0, \dots, x_k] \prod_{i=0}^{k-1} (x - x_i). \quad (4.6)$$

Python – estratto essenziale. Differenze divise e Horner sono implementati direttamente:

```
d = np.asarray(y, dtype=float).copy()
for k in range(1, len(x)):
    d[k:] = (d[k:] - d[k-1:-1]) / (x[k:] - x[:k])

yq = np.full_like(xq, d[-1], dtype=float)
for i in range(len(x) - 2, -1, -1):
    yq = d[i] + (xq - x[i]) * yq
```

Vandermonde, Lagrange e Newton producono lo stesso polinomio sugli stessi nodi; cambiano costruzione, valutazione e comportamento numerico.

4.5 Spline di grado 1: S1

Dispense di Interpolazione, pp. 36 e 38 [1].

È l'interpolazione lineare a tratti, globalmente continua:

$$s^1(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} (x - x_i), \quad x \in [x_i, x_{i+1}]. \quad (4.7)$$

Python – estratto essenziale. `scipy.interpolate.make_interp_spline`, grado 1:

```
model = make_interp_spline(x, y, k=1)
yq = model(xq)
```

4.6 Spline cubica naturale: S3

Dispense di Interpolazione, pp. 38–41 [1].

È una funzione cubica a tratti, di classe C^2 , con derivata seconda nulla agli estremi:

$$s_i^3(x) = c_{1,i} + c_{2,i}(x - x_i) + c_{3,i}(x - x_i)^2 + c_{4,i}(x - x_i)^3, \quad (4.8)$$

$$D^{(2)}s^3(a) = D^{(2)}s^3(b) = 0. \quad (4.9)$$

Python – estratto essenziale. `CubicSpline` con condizione natural:

```
model = CubicSpline(x, y, bc_type="natural", extrapolate=False)
yq = model(xq)
```

4.7 Spline cubica vincolata

Dispense di Interpolazione, pp. 40 [1].

Impone le derivate prime agli estremi:

$$D^{(1)}s^3(a) = f'(a), \quad D^{(1)}s^3(b) = f'(b). \quad (4.10)$$

Nel progetto si stimano queste derivate con polinomi quadratici sui primi e sugli ultimi tre nodi *visibili*.

Python – estratto essenziale. `Polynomial.fit` e condizioni esplicite in `CubicSpline`:

```
left = Polynomial.fit(x[:3], y[:3], 2).deriv()(x[0])
right = Polynomial.fit(x[-3:], y[-3:], 2).deriv()(x[-1])
model = CubicSpline(x, y, bc_type=((1, left), (1, right)),
                    extrapolate=False)
yq = model(xq)
```

4.8 Spline cubica periodica

Dispense di Interpolazione, pp. 40 [1].

È usata per dati periodici con valori compatibili agli estremi:

$$y_0 = y_n, \quad D^{(1)}s^3(a) = D^{(1)}s^3(b), \quad D^{(2)}s^3(a) = D^{(2)}s^3(b). \quad (4.11)$$

Python – estratto essenziale. `CubicSpline` con condizione `periodic`:

```
# The support spans a complete period and y[0] == y[-1].
model = CubicSpline(x, y, bc_type="periodic", extrapolate=False)
yq = model(xq)
```

È verificata nel test sintetico periodico, non forzata sui passaggi stradali non periodici.

4.9 Spline quadratica S2: estensione

La definizione generale delle dispense è $s^m \in C^{m-1}$ (p. 37). S2 è la costruzione aggiuntiva con $m = 2$:

$$s_i^2(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2, \quad s^2 \in C^1. \quad (4.12)$$

Posti $h_i = x_{i+1} - x_i$ e $\delta_i = (y_{i+1} - y_i)/h_i$, si usa

$$a_i = y_i, \quad b_0 = \delta_0, \quad c_i = \frac{\delta_i - b_i}{h_i}, \quad b_{i+1} = b_i + 2c_i h_i. \quad (4.13)$$

La condizione aggiuntiva è $c_0 = 0$.

Python – estratto essenziale. Coefficienti costruiti direttamente e valutazione con `PPoly`:

```
h = np.diff(x)
slopes = np.diff(y) / h
b, c = np.empty_like(h), np.empty_like(h)
b[0] = slopes[0]
for i in range(len(h)):
    c[i] = (slopes[i] - b[i]) / h[i]
    if i + 1 < len(h):
        b[i + 1] = b[i] + 2 * c[i] * h[i]
model = PPoly(np.vstack((c, b, y[:-1])), x, extrapolate=False)
yq = model(xq)
```

4.10 Nodi di Chebyshev

Dispense di Interpolazione, pp. 28–32 [1].

Sono gli zeri dei polinomi di Chebyshev; per grado interpolante al più n si usano gli $n + 1$ zeri di T_{n+1} :

$$T_k(x) = \cos(k \arccos x), \quad x_j = \cos\left(\frac{(2j+1)\pi}{2(n+1)}\right), \quad j = 0, \dots, n. \quad (4.14)$$

Il cambiamento di intervallo è $t_j = (b - a)x_j/2 + (a + b)/2$.

Python – estratto essenziale. Generazione con `numpy.cos`; nel test si valuta la forma di Newton:

```
from src.interpolation import NewtonPolynomial

j = np.arange(n + 1)
x = np.sort(np.cos((2 * j + 1) * np.pi / (2 * (n + 1))))
y = 1 / (1 + 25 * x**2)
yq = NewtonPolynomial(x, y)(xq)
```

Chebyshev è una scelta dei nodi, non una formulazione diversa sugli stessi dati. Il test dedicato usa la funzione di Runge normalizzata.

4.11 Interpolazione trigonometrica

Dispense di Interpolazione, pp. 4 e 6 [1].

È una combinazione di armoniche. Nella forma delle dispense:

$$\tilde{f}(x) = a_{-M}e^{-iMx} + \dots + a_0 + \dots + a_M e^{iMx}. \quad (4.15)$$

Per $N = 2M + 1$ dati, la forma reale con periodo T è

$$p(t) = a_0 + \sum_{k=1}^M [a_k \cos(k\phi(t)) + b_k \sin(k\phi(t))], \quad \phi(t) = \frac{2\pi(t - t_0)}{T}. \quad (4.16)$$

Per $N = 2M$ la somma termina a $M - 1$ e si aggiunge il solo termine $a_M \cos(M\phi(t))$. L'estremo finale duplicato è escluso.

Python – estratto essenziale. Matrice seno/coseno e `numpy.linalg.solve`:

```
N = len(t)
def design(times):
    phi = 2 * np.pi * (times - t0) / T
    columns = [np.ones_like(times)]
    for k in range(1, (N - 1) // 2 + 1):
        columns.extend([np.cos(k * phi), np.sin(k * phi)])
    if N % 2 == 0:
        columns.append(np.cos((N // 2) * phi))
    return np.column_stack(columns)

coefficients = np.linalg.solve(design(t), y)
yq = design(tq) @ coefficients
```

Con campioni mancanti si risolve il sistema sui tempi osservati; non si applica direttamente una FFT a una griglia incompleta.

4.12 Interpolazione razionale: costruzione Floater–Hormann

La famiglia $r(x) = P(x)/Q(x)$ è introdotta a pagina 4 delle dispense. L'implementazione specifica è un'estensione [3]:

$$r(x) = \frac{\sum_{i=0}^n \frac{w_i y_i}{x - x_i}}{\sum_{i=0}^n \frac{w_i}{x - x_i}}, \quad r(x_i) = y_i, \quad (4.17)$$

con $d = \min(3, n)$ e

$$w_i = (-1)^{i-d} \sum_{j=\max(0, i-d)}^{\min(i, n-d)} \prod_{\substack{k=j \\ k \neq i}}^{j+d} \frac{1}{|x_i - x_k|}. \quad (4.18)$$

Python – estratto essenziale. Pesi e rapporto baricentrico calcolati con NumPy:

```
N, d = len(x), min(3, len(x) - 1)
w = np.zeros(N)
for i in range(N):
    for j in range(max(0, i-d), min(i, N-d-1) + 1):
        indices = [k for k in range(j, j+d+1) if k != i]
        w[i] += 1 / np.prod(np.abs(x[i] - x[indices]))
    w[i] *= (-1.0)**(i-d)
w /= np.max(np.abs(w))

def value(z):
    hit = np.flatnonzero(z == x)
    if len(hit):
        return y[hit[0]]
    terms = w / (z - x)
    return (terms @ y) / terms.sum()

yq = np.array([value(z) for z in xq])
```

5 Protocollo di confronto

Il riferimento completo è salvato prima di mascherare i dati. I gap contengono 3, 6, 12 o 24 frame nascosti; ogni fit usa quattro nodi visibili prima e quattro dopo. I campioni nascosti non diventano supporto di altri gap. Gli stessi casi sono utilizzati da tutti i metodi.

```
# Essential scheme from src/benchmark.py
from src.interpolation import Interpolator

visible = mask["visible_indices"]
hidden = mask["hidden_indices"]
model_u = Interpolator(method).fit(times[visible], u[visible])
model_v = Interpolator(method).fit(times[visible], v[visible])
predicted = np.column_stack([
    model_u.predict(times[hidden]),
    model_v.predict(times[hidden]),
])
reference = np.column_stack([u[hidden], v[hidden]])
error = np.linalg.norm(predicted - reference, axis=1)
```

Per il confronto metrico la coppia (u, v) viene sostituita da (E, N) *prima* del fit. Sono ammissibili soltanto i gap per cui tutti i nodi di supporto e i riferimenti nascosti hanno una proiezione valida. La baseline in pixel viene ripetuta su quello stesso sottoinsieme.

Metriche. Si usano ADE e RMS definiti nel Capitolo 2 sugli stessi campioni nascosti e rispetto allo stesso riferimento. Si registrano anche tempi, fallimenti e, per la base monomiale, $\kappa_2(V_n)$. L'errore di ricostruzione in metri rispetto al segnale proiettato non è una misura indipendente dell'accuratezza GPS dell'auto.

6 Risultati disponibili

Il confronto in pixel contiene 5 517 gap, 56 928 campioni nascosti e otto metodi: 44 136 valutazioni, senza fallimenti numerici. Le prime tabelle descrivono il run completo in pixel; il confronto metrico successivo usa il sottoinsieme coperto dalle calibrazioni.

Tabella 2: ADE in pixel con otto nodi visibili.

Metodo	3 frame	6 frame	12 frame	24 frame
S1	0,3005	0,4154	0,6166	0,8799
Vandermonde	0,4362	0,9756	3,1968	14,2928
Lagrange	0,4362	0,9756	3,1968	14,2928
Newton	0,4362	0,9756	3,1968	14,2928
S3 naturale	0,3382	0,5233	0,8315	1,3387
S3 vincolata	0,3382	0,5236	0,8320	1,3402
S2 (estensione)	0,7501	1,1856	1,9437	3,3569
Razionale FH (estensione)	0,6792	1,8519	7,2033	35,1647

Vandermonde, Lagrange e Newton differiscono fra loro al massimo di circa 10^{-10} pixel. I grandi errori sui gap lunghi non si eliminano cambiando soltanto la formulazione dello stesso polinomio.

Tabella 3: Effetto del supporto: ADE per gap di 24 frame, sugli stessi punti nascosti.

Metodo	4 nodi	6 nodi	8 nodi
S1	0,8799	0,8799	0,8799
Polinomio: tre formulazioni	1,1256	3,9149	14,2928
S3 naturale	1,1374	1,3217	1,3387
S3 vincolata	1,1284	1,2934	1,3402
S2	1,5098	2,6194	3,3569
Razionale FH	1,1256	4,8415	35,1647

Più nodi non garantiscono un errore minore. Per i polinomi, 4, 6 e 8 nodi determinano gradi massimi 3, 5 e 7. S1 usa soltanto gli estremi del gap e rimane invariata. La graduatoria dipende anche dalla metrica: per gap di 12 frame S3 naturale ha RMS inferiore a S1, pur avendo ADE maggiore.

6.1 Confronto sul piano stradale calibrato

Le calibrazioni permettono di proiettare 2 186 osservazioni di Sherbrooke e 7 345 di René-Lévesque. Il dominio valido copre una parte delle immagini: **406 gap su 5 517** hanno tutti i supporti e i riferimenti nella zona calibrata; gli altri 5 111 sono esclusi dal confronto metrico.

La baseline in pixel è ricalcolata su questi stessi casi. Per gap di 24 frame, ad esempio, S1 ha ADE 2,0543 pixel nella baseline abbinata e 0,1075 m nel fit metrico. Le grandezze non si convertono mediante un unico fattore: proiezione e interpolazione non sono operazioni commutative. Il riferimento in metri resta la proiezione di YOLO/ByteTrack, non un GPS indipendente dell'automobile.

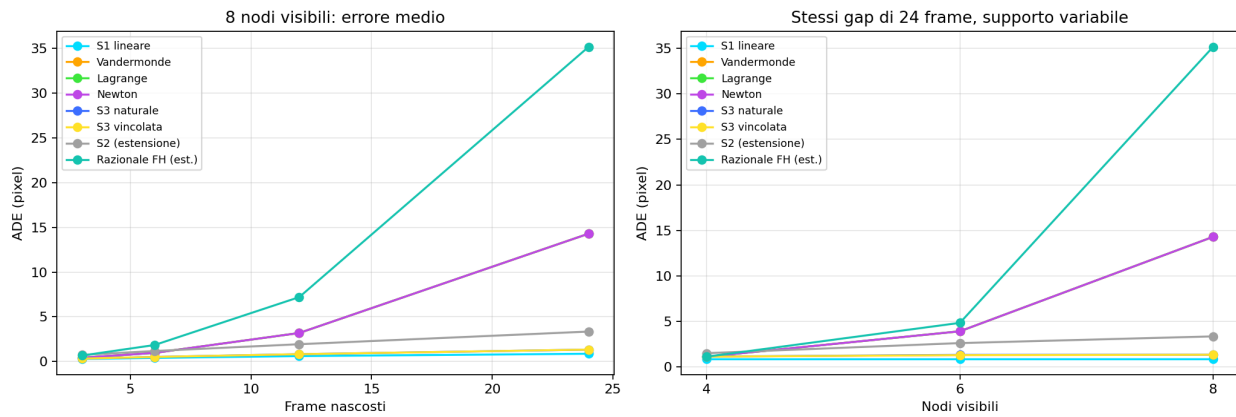


Figura 1: Errori al variare di lunghezza del gap e numero di nodi.

Tabella 4: ADE in metri: interpolazione direttamente nel piano (E, N) , sugli stessi 406 gap ammissibili.

Metodo	3 frame	6 frame	12 frame	24 frame
S1	0,0450	0,0642	0,0878	0,1075
Vandermonde / Lagrange / Newton	0,0749	0,1506	0,5576	2,1673
S3 naturale	0,0570	0,0810	0,1386	0,1987
S3 vincolata	0,0570	0,0810	0,1386	0,1992
S2	0,1231	0,1970	0,3548	0,4885
Razionale FH	0,1166	0,2885	1,2840	5,1797



Figura 2: Zone calibrate in verde e punti usati nel fit. L'arancio indica una corrispondenza esclusa dal fit robusto.

6.2 Esperimenti sintetici

Runge e Chebyshev. Si interpola $f(x) = 1/(1 + 25x^2)$ su $[-1, 1]$. A grado 32, l'errore massimo è circa 5 058,96 sui nodi equispaziati e 0,001402 sulle radici di Chebyshev. La distribuzione dei nodi è determinante.

Periodicità. Si usa, con $T = 6$ secondi,

$$P(t) = \begin{bmatrix} 12 \cos(2\pi t/T) + 1.5 \cos(4\pi t/T) \\ 8 \sin(2\pi t/T) \end{bmatrix}. \quad (6.1)$$

Si nascondono otto campioni fra 49, in tre collocazioni. S3 periodica raggiunge ADE di circa 0,0274–0,0379; la trigonometrica errori dell'ordine di 10^{-9} , perché il segnale appartiene alla base scelta. Queste unità sintetiche non vengono mescolate ai risultati stradali.

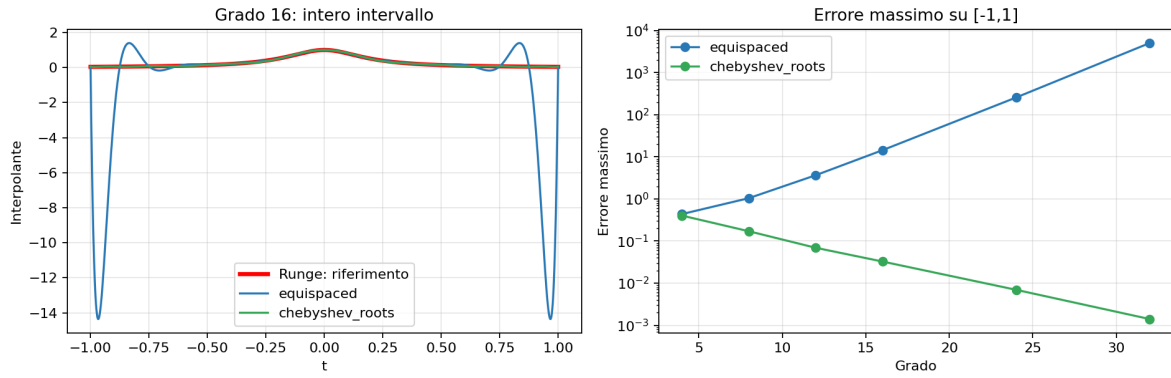


Figura 3: Funzione di Runge e scelta dei nodi.

6.3 Clip dei passaggi

Sono disponibili 12 clip separate, di circa 6,4–8 secondi. Il riferimento YOLO/ByteTrack è rosso acceso; otto riquadri affiancati mostrano gli interpolatori con colori distinti. Zoom e istanti nascosti sono uguali per tutti. La selezione usa movimento e distribuzione temporale, non gli errori ottenuti. Codifica GPU con NVIDIA NVENC.

I video completi con la ricostruzione sono:

- [Urban René-Lévesque](#) (8501 frame, circa 4 min 44 s);
- [Urban Sherbrooke](#) (4000 frame, circa 2 min 13 s).

L'indice delle clip brevi e i relativi file sono nella directory `reports/generated/course/clips/`.

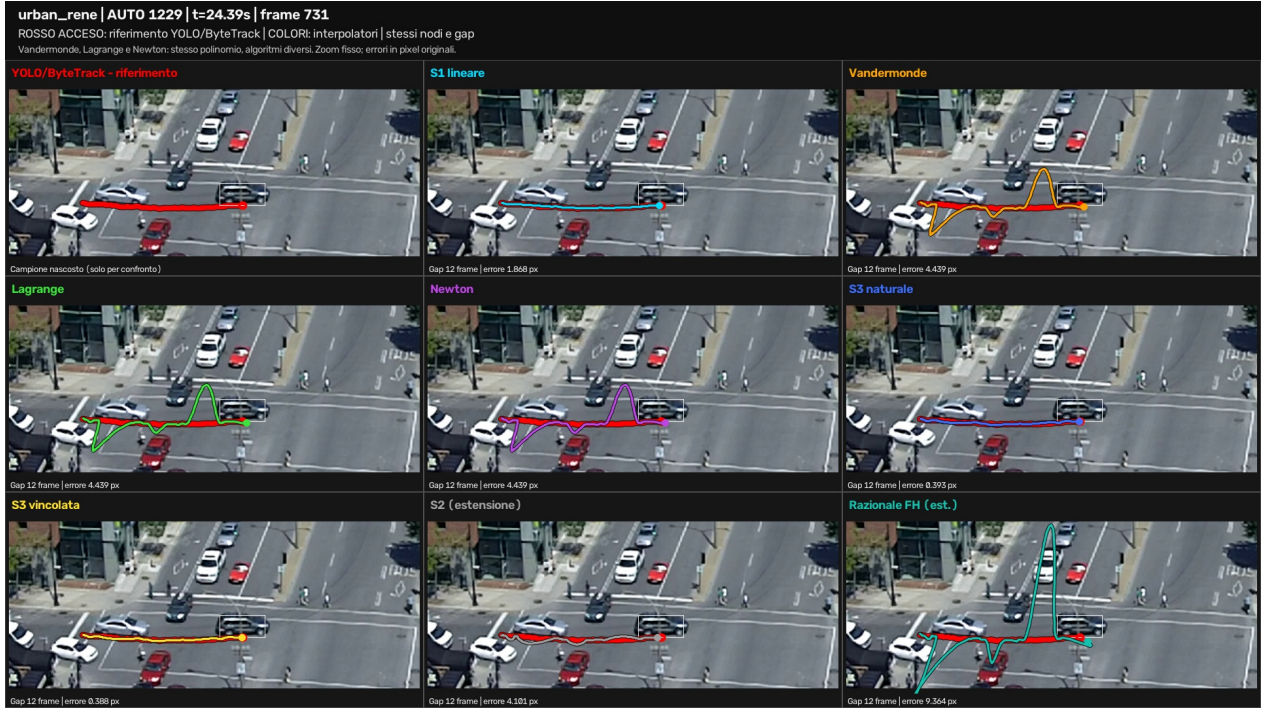


Figura 4: Stesso passaggio, riferimento rosso e otto interpolatori. Immagini Urban Tracker, elaborazione del progetto.

7 Cinematica

Velocità, accelerazione e heading si ricavano dagli stessi modelli interpolanti usati per le coordinate $E(t)$ e $N(t)$:

$$\mathbf{v}(t) = \begin{bmatrix} \dot{E}(t) \\ \dot{N}(t) \end{bmatrix}, \quad V(t) = \sqrt{\dot{E}(t)^2 + \dot{N}(t)^2}, \quad \mathbf{a}(t) = \begin{bmatrix} \ddot{E}(t) \\ \ddot{N}(t) \end{bmatrix}. \quad (7.1)$$

L'heading, in gradi orari dal Nord, è

$$\theta(t) = \left[\frac{180}{\pi} \text{atan2}(\dot{E}(t), \dot{N}(t)) + 360 \right] \bmod 360. \quad (7.2)$$

Quindi non si costruisce un interpolatore separato per la velocità o per l'heading: si interpola la posizione, si derivano i due modelli e si calcolano velocità e heading dalle derivate. In particolare, la velocità è il modulo della derivata e l'heading è l'angolo del vettore velocità; quest'ultimo va escluso o marcato non definito quando la velocità è troppo bassa. Il metodo di interpolazione scelto per E, N determina pertanto anche la regolarità delle grandezze cinematiche. Il codice dispone già delle derivate e applica la regola della catena:

$$\frac{d}{dt} = \frac{2}{t_{\max} - t_{\min}} \frac{d}{d\tau}, \quad \frac{d^2}{dt^2} = \left(\frac{2}{t_{\max} - t_{\min}} \right)^2 \frac{d^2}{d\tau^2}. \quad (7.3)$$

Schema Python della fase futura, non risultato già misurato:

```
model_E = Interpolator(method).fit(t_visible, E_visible)
model_N = Interpolator(method).fit(t_visible, N_visible)
vE = model_E.derivative(tq, order=1)
vN = model_N.derivative(tq, order=1)
aE = model_E.derivative(tq, order=2)
aN = model_N.derivative(tq, order=2)
speed_mps = np.hypot(vE, vN)
heading = (np.degrees(np.arctan2(vE, vN)) + 360) % 360
heading[speed_mps < speed_min] = np.nan
```

La valutazione è stata eseguita sui 406 gap metrici già validi. Per ogni gap si interpolano soltanto i punti visibili, si calcolano le derivate in tempo fisico e si confrontano velocità, accelerazione e heading con una stima ottenuta mediante differenze finite sulla traiettoria completa proiettata. Quest'ultima è una pseudo-ground-truth basata su YOLO/ByteTrack, non una misura indipendente GPS o inerziale. L'heading è valutato solo per velocità di riferimento almeno pari a 0,5 m/s.

Tabella 5: Errore cinematica per gap di 24 frame, media sui 406 gap metrici.

Metodo	MAE velocità (m/s)	MAE accelerazione (m/s ²)	MAE heading (gradi)
S1	0.7607	19.5135	24.2816
S3 naturale	0.9262	20.0055	26.9254
S3 vincolata	0.9276	20.0034	26.9037
S2 (estensione)	1.4928	21.6733	35.2215
Vandermonde / Lagrange / Newton	8.7811	90.9516	69.3162
Razionale FH (estensione)	27.1535	271.3230	81.0957

I risultati completi per tutti i gap sono in `data/urbantracker/results/ground/KINEMATICS.md` e `kinematic_summary.csv`. In questo esperimento S1 è il miglior metodo anche per velocità e heading sui gap lunghi e presenta il minore errore di accelerazione nella tabella; S3 naturale e S3 vincolata sono alternative regolari con errore vicino. I metodi polinomiali globali Vandermonde, Lagrange e Newton sono equivalenti e amplificano le oscillazioni nelle derivate, mentre Floater–Hormann è il peggiore sui gap lunghi. Queste conclusioni valgono per questa pseudo-ground-truth e per le scene analizzate.

8 Riproduzione e conclusioni

I principali comandi, dalla directory principale della repository, sono:

```
python3 -m src.benchmark
python3 -m src.course_experiments
python3 -m src.passage_clips
python3 -m src.course_report --verify-clips

python3 -m src.calibrate_ground
# Dopo avere scelto i punti e premuto Pubblica:
python3 -m src.project_ground
python3 -m src.metric_benchmark
python3 -m src.kinematic_benchmark
python3 -m src.ground_report

python3 -m unittest discover -s tests -v
```

Il progetto ha già verificato interpolazione, equivalenza dei polinomi, assenza di valori nascosti nel fit e integrità delle clip. L'integrazione geometrica è verificata su casi sintetici: ordine Est/Nord, trasformazioni geografiche, compatibilità e dominio valido.

I risultati disponibili mostrano che **un metodo più complesso non è automaticamente più accurato**. Per queste scene, sui gap di 3, 6, 12 e 24 frame, **S1 lineare è il metodo migliore per ADE e RMS** ed è anche il più economico tra i metodi con errore comparabile. La **S3 naturale** è la principale alternativa: segue S1 con errori contenuti e fornisce una traiettoria più regolare, ma con costo maggiore. S3 vincolata ha risultati quasi identici; Vandermonde, Lagrange e Newton sono tre formulazioni dello stesso polinomio e quindi non costituiscono metodi diversi in termini di accuratezza. S2 e soprattutto la razionale Floater–Hormann risultano peggiori su questi video, in particolare per gap lunghi. Questa graduatoria è confermata anche dalla valutazione cinematica disponibile: S1 è il compromesso migliore per posizione, velocità, heading e accelerazione sui gap lunghi. S3 naturale resta una valida alternativa quando si privilegia la regolarità della traiettoria.

I due video completi e le dodici clip sono riproducibili dai link nella sezione precedente e dai comandi sopra. Le osservazioni restano riferimenti YOLO/ByteTrack, le identità non sono manualmente validate e i gap possono condividere nodi visibili: le conclusioni sono descrittive per queste scene.

Riferimenti bibliografici

- [1] S. Boscarino, *Analisi Numerica – Interpolazione*, dispense del corso, 28 aprile 2025, file `Interpolazione.pdf`.
- [2] J.-P. Jodoin, G.-A. Bilodeau, N. Saunier, *Urban Tracker: Multiple Object Tracking in Urban Mixed Traffic*, WACV 2014. <https://www.jpjodoin.com/urbantracker/dataset.html>.
- [3] M. S. Floater, K. Hormann, *Barycentric rational interpolation with no poles and high rates of approximation*, Numerische Mathematik 107, 315–331, 2007.
- [4] NumPy e SciPy, documentazione di algebra lineare e interpolazione: <https://numpy.org/doc/stable/>, <https://docs.scipy.org/doc/scipy/reference/interpolate.html>.